

Python Mastery: Paramiko

SSH, SFTP, and Remote Server Automation

1. Introduction to Paramiko

Paramiko is a pure-Python implementation of the SSHv2 protocol. It provides both client and server objects. It is the core library used by higher-level tools like Fabric and Ansible for SSH connectivity.

Installation

Install Paramiko using pip:

```
pip install paramiko
```

2. SSH Client Connections

The SSHClient class is the primary interface for making SSH connections to a remote server.

```
import paramiko

ssh = paramiko.SSHClient()
# Automatically add host keys from the remote server
ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())

ssh.connect(hostname='192.168.1.10', username='admin', password='secret')

# Execute a command
stdin, stdout, stderr = ssh.exec_command('ls -l')
print(stdout.read().decode())

ssh.close()
```

3. SFTP for File Transfers

Paramiko also provides an SFTP client to transfer files over the encrypted SSH connection.

```
ssh = paramiko.SSHClient()
ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
ssh.connect('192.168.1.10', username='admin', password='secret')

sftp = ssh.open_sftp()
# Upload a file
sftp.put('local_file.txt', '/remote/path/remote_file.txt')
# Download a file
sftp.get('/remote/path/remote_file.txt', 'downloaded_file.txt')

sftp.close()
ssh.close()
```

4. Authentication with SSH Keys

Using passwords in scripts is often insecure. Paramiko supports authentication using private SSH keys.

```
key = paramiko.RSAKey.from_private_key_file('/path/to/id_rsa')
ssh.connect(hostname, username=user, pkey=key)
```

5. Error Handling

When automating remote tasks, it is crucial to handle potential connection issues using exceptions like `AuthenticationException` or `SSHException`.